If you're curious, you could compare this result with what would happen when updating a ConfigMap without Kustomize. Kubernetes would update the volume mounted to the Catalog Service container, but the application would not be restarted and would still return the old value.

NOTE Depending on your requirements, you might need to avoid a rolling restart and have the applications reload their configuration at runtime. In that case, you can disable the hash suffix strategy with the `disableNameSuffixHash: true` generator option and perhaps rely on something like Spring Cloud Kubernetes Configuration Watcher to notify the applications whenever a ConfigMap or Secret is changed.

When you're done experimenting with the Kustomize setup, you can stop the port-forwarding process (Ctrl-C) and undeploy Catalog Service (`kubectl delete -k k8s`).

Since we moved from plain Kubernetes manifests to Kustomize, we still need to update a couple of things. In chapter 7, we used Tilt to achieve a better development workflow when working locally on Kubernetes. Tilt supports Kustomize, so we can configure it to deploy applications via a `Kustomization` resource rather than via plain Kubernetes manifests. Go ahead and update the Tiltfile in your Catalog Service project as follows.

Listing 14.8 Configuring Tilt to deploy Catalog Service using Kustomize

```
custom_build(
    ref = 'catalog-service',
    command = './gradlew bootBuildImage --imageName $EXPECTED_REF',
    deps = ['build.gradle', 'src']
)

k8s_yaml(kustomize('k8s'))      ❶

k8s_resource('catalog-service', port_forwards=['9001'])
```

❶ Runs the application from the Kustomization located in the k8s folder